Gymnázium Brno, Vídeňská, příspěvková organizace



Ročníkový projekt
pro Programování

Kvízová aplikace s editorem GvidLearn

24. května 2025

Jakub Vávra, 5. G

Abstrakt

Tato technická zpráva slouží jako dokumentace k ročníkovému projektu. Cílem projektu bylo vytvořit kvízovou aplikaci s grafickým uživatelským rozhraním v jazyce Java. V rámci rozšíření je součástí aplikace také editor.

Klíčová slova

Java, grafické uživatelské rozhraní pomocí knihovny Swing, kvízová aplikace, ročníkový projekt

Citace

Jakub Vávra: *Kvízová aplikace s editorem GvidLearn*, ročníkový projekt, Brno, Gymnázium Brno, Vídeňská, příspěvková organizace, 2025.

Kvízová aplikace s editorem GvidLearn

Prohlášení

Prohlašuji, že jsem tuto práci vytvořil samostatně a poctivě jsem uvedl všechny zdroje, ze kterých jsem vycházel.

.....

Jakub Vávra

24. května 2025

© Jakub Vávra, 2025.

Tato práce vznikla jako školní dílo na škole Gymnázium Brno, Vídeňská, příspěvková organizace. Práce je chráněna autorským zákonem a její užití bez udělení oprávnění autorem je nezákonné, s výjimkou zákonem definovaných případů. Autor souhlasí s archivací této práce a s jejím použitím pro studijní účely na výše zmíněné škole a jejích případných právních nástupcích.

Obsah

1 Úvod **2**

2 Analýza a návrh řešení **3**

2.1 Specifikace zadání 3

2.2 Analýza problémů 4

3 Popis řešení **6**

3.1 Uchování dat 6

3.2 Zpracování souboru a ošetření chyb 6

3.3 Implementace GUI 9

3.4 Implementace rozšíření 9

3.5 Uživatelská dokumentace 12

4 Závěr **14**

A Použité technologie **15**

Kapitola 1

Úvod

Cílem tohoto dokumentu je popsat mé řešení ročníkového projektu pro předmět Programování. Zadáním projektu bylo vytvořit aplikaci s grafickým uživatelským rozhraním[1] v jazyce Java. Aplikace slouží ke spouštění výukových kurzů na různá témata. Jako vlastní rozšíření jsem vytvořil editor kurzů, který je integrován přímo do aplikace a disponuje funkčním GUI.

Projekt měl za cíl rozvíjet několik schopností. Tou nejdůležitější bylo procvičení konceptů objektově orientovaného programování [2] a to těch základních, jako je dědičnost a polymorfismus, ale i pokročilých, například návrhové vzory. Dále to byla schopnost rozdělit si práci na několik etap, tak aby bylo zpracovávání projektu pohodlné a časově zvládnutelné. V neposlední řadě zadání apelovalo na uplatnění vlastní kreativity při návrhu rozšíření.

[1] Dále jako GUI, z anglického *Graphical User Interface*
[2] Dále jako OOP

Kapitola 2

Analýza a návrh řešení

2.1 Specifikace zadání

Cílem projektu je vytvořit výukovou (kvízovou) aplikaci v jazyce Java s GUI. Aplikace bude schopna zobrazovat kurzy, které obsahují různá témata s výukovými materiály a sadou otázek k procvičení. Zobrazování obsahu kurzu má svá specifika: program zobrazí další téma pouze po zodpovězení všech otázek k předchozímu tématu, uživatel musí mít vždy možnost přerušit test a zobrazit si poslední téma (toto ale vyvolá resetování postupu) a po zodpovězení otázek zobrazuje aplikace skóre. Obsah kurzu je uložen v textovém souboru, přičemž každý kurz má vlastní soubor. Formát souboru je pevně zadaný a blíže přiblížen v sekci 2.1.

Součástí zadání je také návrh a implementace vlastních rozšíření. Zadání specifikuje několik možných rozšíření, ze kterých jsem realizoval dvě. Prvním je vytvoření nového typu otázky – Pravda/lež. Druhým je editor témat a otázek, toto zadání jsem rozšířil a editor vztáhl také na samotný kurz.

Formát souboru

Zadání stanovuje formát souboru s kurzy. Jednotlivé části kurzu jsou odděleny klíčovými slovy[1]. Kurz lze rozdělit logicky na tři části: kurz, témata a otázky. Pro kurz platí, že musí obsahovat název kurzu (`Kurz:`) a alespoň jedno téma. Téma musí obsahovat název tématu (`Téma:`), výukový materiál a alespoň jednu otázku (`Test:`). Otázky mají různé parametry a každá začíná vlastním klíčovým slovem, které odpovídá typu otázky (`Text:, Doplňovačka:, PravdaLež:`). Kdekoli v souboru se také může vyskytnout znak `#`, který značí komentář, tedy řádek, který program nezpracovává. Ukázka vstupního souboru je obsahem obrázku 2.1

[1] Jednotlivá slova budou dále psána v závorce za částí souboru, kterou oddělují.

```
Kurz:Biologie01
# Úvodní kurz mikrobiologie
Téma:Buňka
Buňka (lat. cellula) je základní stavební
a funkční jednotka těl živých organismů,
nikoliv však těch nebuněčných, jako jsou
viry, viroidy a virusoidy.

Test:
Doplňovačka:Buňky netvoří organismy
jako _, _ a _.
Body:5
OK:viry,viroidy,virusoidy
KO:prvoci,řasy,bakterie

Text:Latinský název pro buňku je
Body:2
OK:Cellula

PravdaLež:Buňka je základní stavební
a funkční jednotka nebuněčných organismů.
Body:1
OK:lež
Tvrzení:Buňka je základní stavební
a funkční jednotka buněčných organismů.
```

Obrázek 2.1: Ukázka vstupního souboru.

2.2 Analýza problémů

V zadání je také popsáno několik etap, které rozdělují postup řešení na několik problémů. S ohledem na toto rozdělení jsem své řešení rozdělil na několik dílčích částí.

Problém 1 – uchování dat

Jak je nastíněno v popisu vstupního souboru, obsah kurzu lze pomocí klíčových slov rozdělit na několik částí. Prvním problémem je tedy návrh vhodných tříd pro uchování dat tak, aby se dal kurz snadno rozšířit, například o nový typ otázek.

Problém 2 – zpracování souboru a ošetření chyb

Pro zpracování souboru je nutné zvolit vhodný způsob, který by umožnil během čtení vyhodnocovat klíčová slova. Zároveň je nutné načítání vhodně rozdělit do tříd, které odpovídají struktuře kurzu. Třídy pro načítání musí být schopné identifikovat chyby v souboru a ošetřit je pomocí výjimek.

Problém 3 – návrh GUI

GUI by mělo být navrženo v souladu se strukturou kurzu. Zároveň by propojení GUI se zbytkem projektu mělo respektovat princip oddělení odpovědnosti, aby program zůstal snadno rozšiřitelný. Dále musí být zachována specifika zobrazování obsahu, který je popsán zadáním, jako je postupné zobrazování otázek nebo možnost ukončit test a zobrazit téma.

Problém 4 – návrh rozšíření

Jako rozšíření jsem si zvolil přidání nového typu otázky Pravda/Lež. Ta bude implementována jako ostatní typy otázek.

Druhým rozšířením bude editor otázek. Editor musí být schopen upravovat existující kurzy a vytvářet nové. Editor bude integrován do GUI aplikace. Důležité je, aby upravované kurzy byly bez problémů znovu spustitelné. Tedy musí být zajištěno, aby program provedl správný zápis kurzu do souborů a aby uživatel nemohl kurz, který je v rozporu s formátem souboru.

Rozhraní editoru nemůže být přístupné bez přihlášení, proto bude zamčené pro normální uživatele, kteří budou moci kurzy pouze spouštět.

Kapitola 3

Popis řešení

3.1 Uchování dat

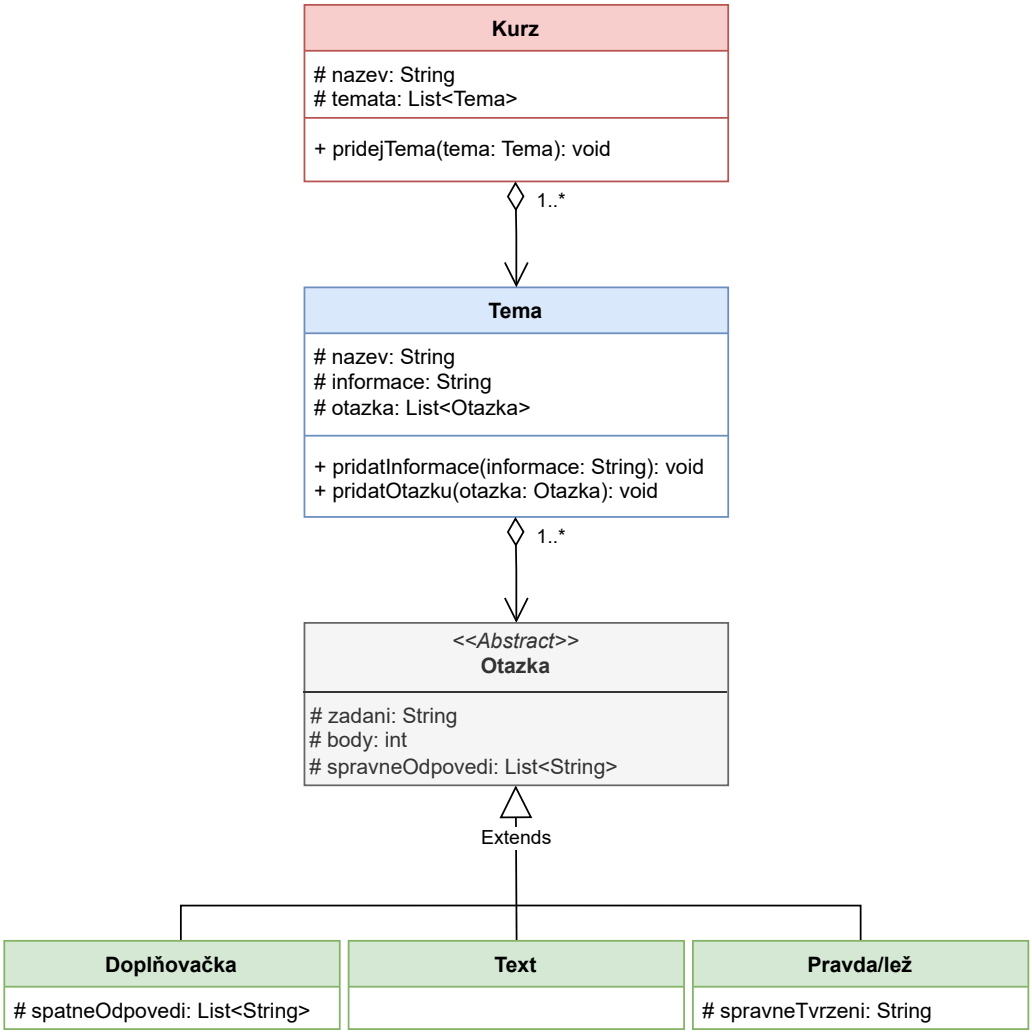
Systém tříd pro uchování popisuje UML diagram 3.1. Třídy kopírují logické rozdělení, které jsem popsal v sekci s popisem vstupního souboru 2.1. Třídu *Otazka* jsem implementoval jako abstraktní třídu. Nabízí se i implementace skrze rozhraní, které jsem nepoužil, protože má abstraktní třída *Otazka* obsahuje také seznamy s klíčovými slovy souboru, které jsou dále využívány při načítání souboru.

3.2 Zpracování souboru a ošetření chyb

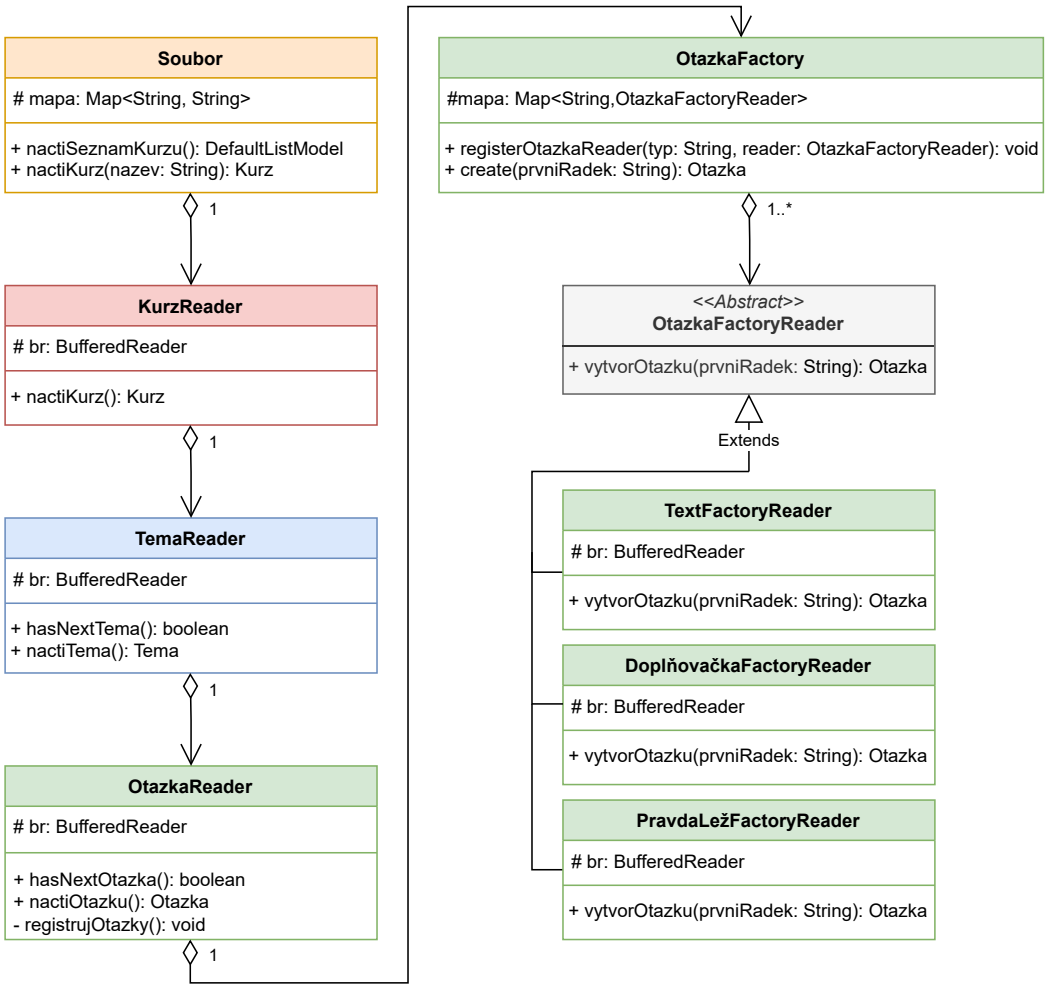
Třídy realizující zpracování souboru jsou popsány v diagramu 3.2. Celé načítání obaluje třída *Soubor*, která obsahuje kolekci s názvy kurzů a odpovídajícími cestami k souborům. Tyto údaje jsou uloženy v souboru *Kurzy.txt*. Ostatní třídy odpovídají třídám pro uchování dat. Pro načítání otázek jsem zvolil návrhový vzor tovární třída, který je realizován ve třídě *OtazkaFactory*. Třídy, které zajišťují načítání jednotlivých otázek, jsou potomky abstraktní třídy *OtazkaFactoryReader*. Tato třída by šla nahradit rozhraním, ale stejně jako abstraktní třída *Otazka* obsahuje seznamy s klíčovými slovy.

Pro načítání ze souboru jsem zvolil třídu *BufferedReader*, která disponuje metodou *readLine(): String*, která umožňuje načítání celého řádku. V určitých částech programu je nutné číst vícekrát řádky s klíčovými slovy, proto jsem také využil metody *mark(readAheadLimit: int): void* a *reset(): void*, které opětovné čtení umožňují.

Vytvořil jsem také systém výjimek, které program dokáže zpracovat. Třídy *Kurz, Tema a Otazka* vyhazují vždy vlastní hlavní výjimku. Dodatečně každá z tříd disponuje vedlejšími výjimkami, které popisují různé chybové stavy, například chybějící atribut. Tyto výjimky jsou poté předány hlavní výjimce parametrem *cause*.



Obrázek 3.1: Třídy pro uchování dat.



Obrázek 3.2: Třídy pro načítání dat.

3.3 Implementace GUI

GUI jsem implementoval pomocí standardní knihovny Swing, která je součástí JDK[1]. Dále jsem použil knihovnu FlatLaf, což je rozšiřující knihovna pro Swing, která upravuje vzhled GUI.

Veškeré GUI aplikace, kromě dialogových oken, se odehrává ve framu *mainFrame*. Do tohoto framu mohou být přidány různé panely, jak je ukázáno v diagramu 3.3. V panelu *KurzyPanel* si uživatel vybere z nabídky kurzů pomocí komponentu typu JList. Panel *TemaPanel* zobrazí výukový materiál tématu a obsahuje tlačítko pro spuštění testu a pro vrácení se na *KurzyPanel*. Panel *PanelOtazky* zobrazuje otázky, mezi kterými se lze posouvat tlačítky. Také obsahuje tlačítko pro zobrazení tématu. Do panelu jsou generovány panely jednotlivých otázek pomocí tovární třídy *PanelFactory*. Panely s otázkami jsou implementovány rozdílně, tak aby odpovídaly struktuře otázky. Po zodpovězení otázek tématu a po zodpovězení všech otázek v kurzu se zobrazí odpovídající panel s výsledky.

Spojení GUI s třídami pro uchování dat je vytvořeno pomocí statických metod ve třídě projektu. V této třídě jsou kurzy načteny a uloženy k použití. Tato třída také obsahuje kolekci objektů *Vysledek*, kde jsou uchovávány informace o skóre v testech.

3.4 Implementace rozšíření

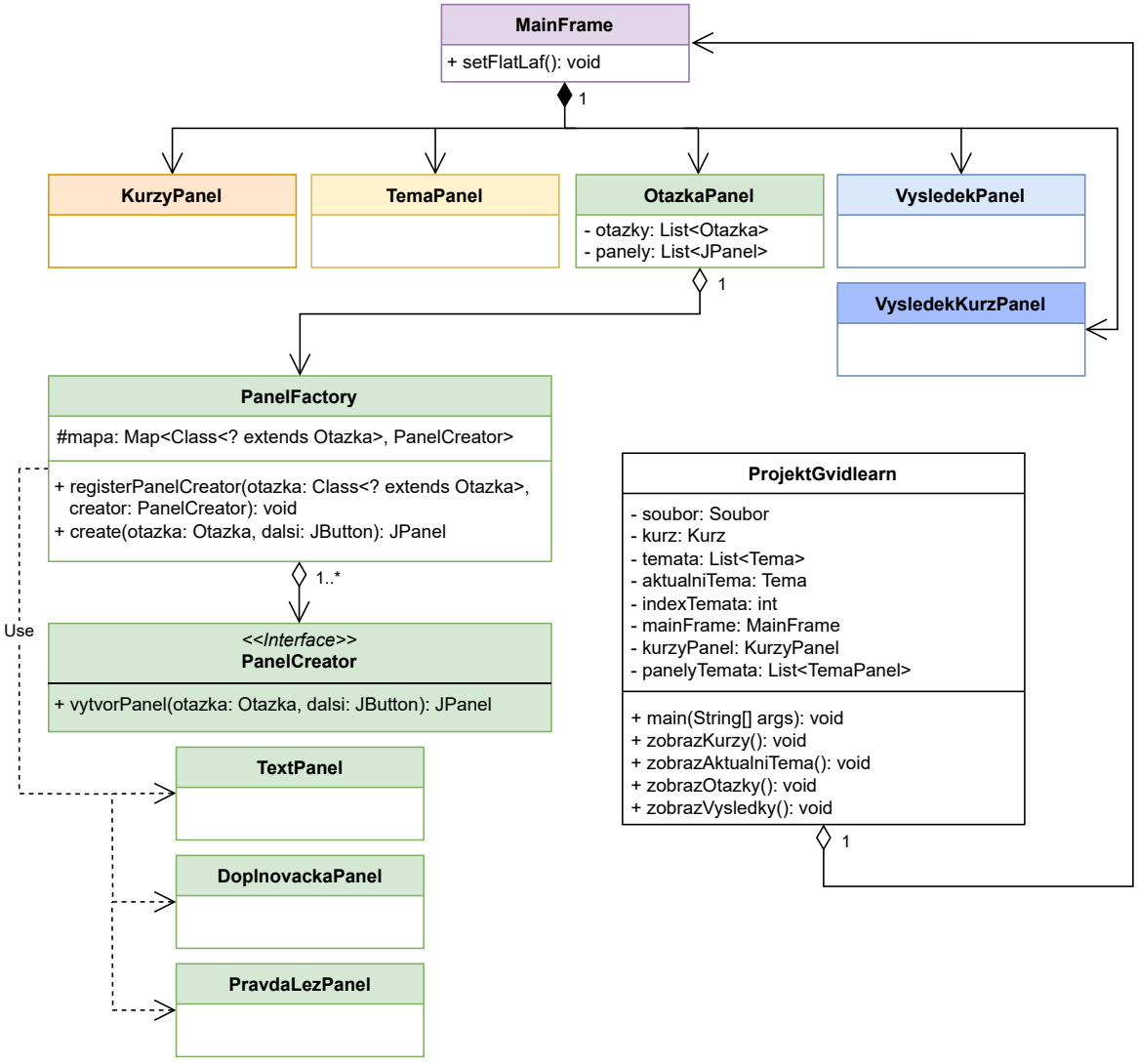
Diagram 3.4 ukazuje schéma rozšíření. Jádrem editoru je třída *Editor*, která umožňuje korektní zápis nových a upravených kurzů do souborů. Rovněž také obsahuje metody pro kontrolu obsahu kurzů. Tuto třídu používají panely *EditorNovyPanel*, který slouží pro vytváření nových kurzů, a *EditorStavajiciPanel*, který umožňuje úpravu existujících kurzů. Oba tyto panely používají stejné panely implementující rozhraní *GUIInterface*, ale používají je rozdílným způsobem.

To se projevuje zejména u panelu *RozhodovaciPanel*, který obsahuje tlačítka s možnostmi pro vytváření nových částí kurzu. Tyto tlačítka používají metody z rozhraní *EditorInterface*, které implementují oba hlavní panely. Rozhodovací panel obsahuje atribut typu *EditorInterface*, jehož metody používá. Zároveň panel také kontroluje, aby uživatel mohl použít vždy jen ta tlačítka, která dávají smysl s ohledem na strukturu kurzu.

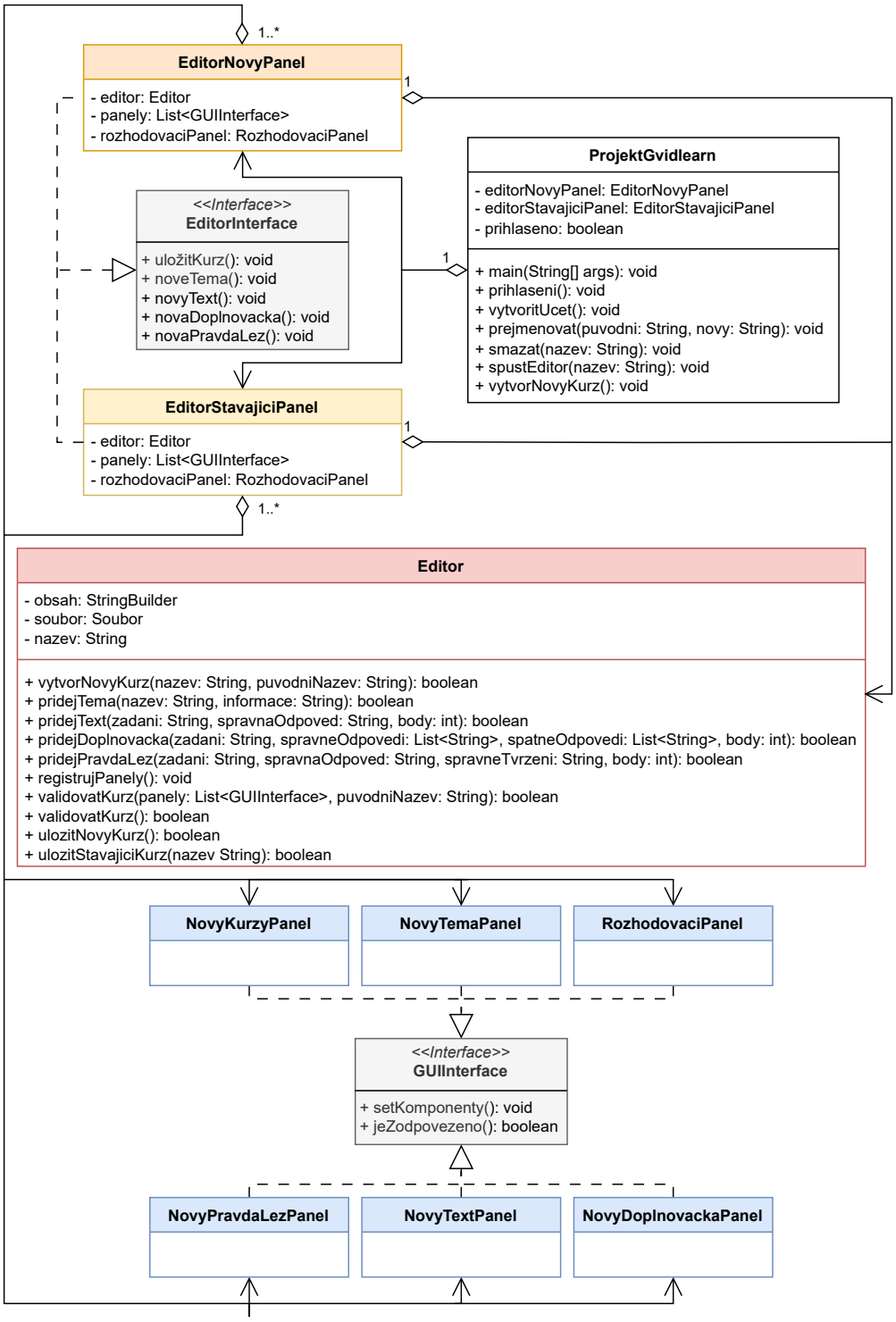
V diagramu nejsou vyznačeny dvě továrny, které jsou používány při upravování existujících kurzů. Konkrétně se jedná o třídu *NovyPanelFactory*, která je použita v panelu *EditorStavajiciPanel* pro převod dat na panely a o třídu *ZpracujPanelFactory*, kterou používá třída *Editor* a slouží naopak pro převod panelů na data.

Editor je spouštěn z panelu *KurzyPanel* a je propojen statickými metodami s třídou projektu. V této třídě jsou také implementovány metody na smazání a přejmenování kurzů. Pro používání funkcí editoru je nutné se přihlásit k admin účtu. Přihlašování a vytváření nových admin účtů je rovněž implementováno ve třídě projektu. Přihlašování je implementováno pomocí dialogových oken *JOptionPane* z knihovny Swing.

[1] Java Development Kit



Obrázek 3.3: Schéma základního GUI.



Obrázek 3.4: Implementace editoru.

3.5 Uživatelská dokumentace

Po spuštění aplikace si uživatel může vybrat z nabídky kurzů a spustit kurz tlačítkem *Spustit kurz*. Poté se budou postupně zobrazovat jednotlivá témata kurzu. Po prostudování tématu spustí uživatel tlačítkem *Spustit test* otázky. Případně se může tlačítkem *Zpět na kurzy* vrátit na domovskou obrazovku. Po spuštění testu se zobrazí zamíchané otázky mezi kterými se lze pohybovat tlačítky *Předchozí* a *Další*. Pro zkontrolování otázky je třeba použít tlačítko *Zkontrolovat*. Po stisknutí tlačítka se objeví dialogové okno s výsledkem odpovědi. Po zodpovězení všech otázek se objeví výsledky daného testu. Uživatel se může vrátit na téma tlačítkem *Zpět na téma* nebo pokračovat dalším tématem tlačítkem *Další*. Po zodpovězení všech otázek v kurzu se objeví vyhodnocení za celý kurz, odkud je možné vrátit se na domovskou obrazovku tlačítkem *Zpět na kurzy*.

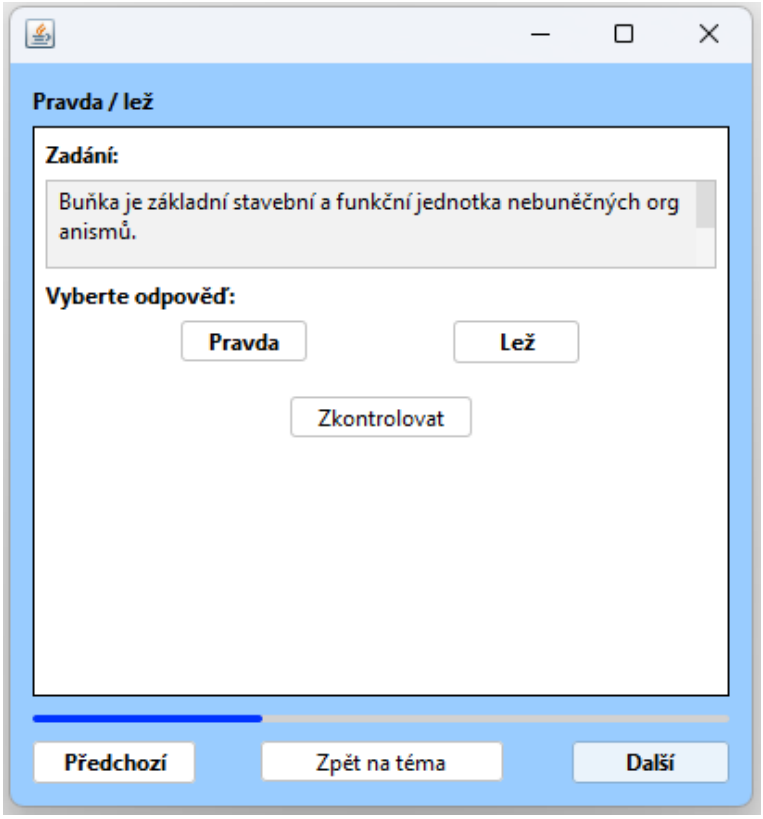
Na domovské obrazovce jsou také tlačítka pro editování kurzů. Ty jsou po spuštění zamčená a pro jejich používání je třeba se přihlásit. To lze udělat po stisknutí ikonky v levém dolním rohu, kde se uživatel může přihlásit nebo vytvořit nový účet. Poté lze používat funkce editoru. Tlačítko *Smazat kurz* umožňuje odstranit vybraný kurz. Tlačítko *Přejmenovat kurz* poté slouží k přejmenování.

Tlačítkem *Vytvořit nový kurz* spustí uživatel prostředí pro vytváření nových kurzů. Uživatel si může vybrat různé části kurzu, které budou postupně přidávány. Přidání nové části provede tlačítko *Potvrdit*, které zároveň zkontroluje zadávané údaje. Kurz lze uložit jen pokud má správný formát a zároveň nelze přidávat části, které by byly v rozporu s formátem kurzu. Kurz je uložen po stisknutí tlačítka *Uložit nový kurz*. Uživatel může editor kdykoliv opustit tlačítkem *Zpět na kurzy*, poté bude ale nový kurz ztracen. Toto je preferovaný způsob přidávání kurzů. Pokud by chtěl uživatel přidat vlastní soubor, musí jej přidat do složky projektu a zapsat do souboru Kurzy.txt.

Tlačítko *Upravit kurz* umožňuje editovat existující kurzy. Po stisknutí se uživateli zobrazí podobný editor, jako při vytváření nového kurzu. Uživatel může mezi částmi kurzu procházet tlačítky *Zpět* a *Dále*. Jednotlivé části může odstranit tlačítkem *Smazat* nebo upravit tlačítkem *Upravit*. Poté musí upravenou část zkontrolovat tlačítkem *Potvrdit*. Pro vkládání nových částí a ukládání kurzu slouží tlačítko *Nový/uložit*. Upravený kurz musí splňovat podmínky na formát kurzu, což je kontrolováno před uložením kurzu.



Obrázek 3.5: Domovská obrazovka aplikace.



Obrázek 3.6: Ukázka spuštěného kurzu.

Kapitola 4

Závěr

Vytvořil jsem funkční aplikaci s GUI, která může sloužit jako výuková aplikace pro libovolný předmět. Tím jsem splnil podmínky zadání. Jako hlavní rozšíření jsem přidal editor, který umožňuje pohodlnou manipulaci s kurzy přímo v rámci aplikace. V budoucnu bych chtěl vytvořit rozšíření, které by umožnilo používat aplikaci v operačním systému Android. Díky tomuto projektu jsem si vyzkoušel návrh větší aplikace a naučil se vytvářet GUI pro konzolové aplikace. Myslím si, že se jedná o užitečnou aplikaci, kterou lze použít v rámci přípravy na testy či zkoušení do školy.

Dodatek A

Použité technologie

Verze JDK: JDK 21
Vývojové prostředí: Apache Netbeans 24
Textový procesor: LaTeX
Program pro vytváření UML diagramů: draw.io

Při zpracování projektu jsem používal Výukového chatbota pro programování, kterého jsem vytvářel v rámci Středoškolské odborné činnosti. Chatbot je dostupný na platformě ChatGPT.